

RĪGAS TEHNISKĀ UNIVERSITĀTE
DATORZINĀTNES, INFORMĀCIJAS TEHNOLOĢIJAS
UN ENERĢĒTIKAS FAKULTĀTE
FOTONIKAS, ELEKTRONIKAS UN ELEKTRONISKO
SAKARU INSTITŪTS

9.Laboratorijas darbs

MCU

Bakalaura studiju programmas
“Viedās elektroniskās sistēmas”

2. kursa RECV0 2.grupas students,
Rīks Sipovičs

Studenta apliecības Nr. 241REC061

Rīga 2026

```

OR  R0, R0      ;0x0090 tukša pirmā instrukcija, lai vieglāk skaitīt rindiņas

LDR  R0, 0x01   ;0x3000 R0 = pirmais skaitlis no RAM

LDR  R1, 0x02   ;0x3101 R1 = otrais skaitlis no RAM

LDI  R2, 0      ;0x2200 Notīra R2

LDI  R3, 0      ;0x2300 Notīra R3

LOOP1:  OR  R0, R0      ;0x0090 pārbauda vai R0 == 0

        BREQ RES      ;0x650c ja R0 == 0 -> pārlec uz RES

        ADD  R2, R1     ;0x0201 R2 = R2 + R1

        BRCC SKP_INC   ;0x600a ja Carry nav -> skip

        INC  R3         ;0x0320 ja bija Carry -> ++

SKP_INC: DEC  R0        ;0x0030 R0--

        JMP  LOOP1     ;0x5005 atkārtot ciklu

RES:    STR  R2, 0x03   ;0x4202 RAM R2 = LOW byte

        STR  R3, 0x04   ;0x4303 RAM R3 = HIGH byte

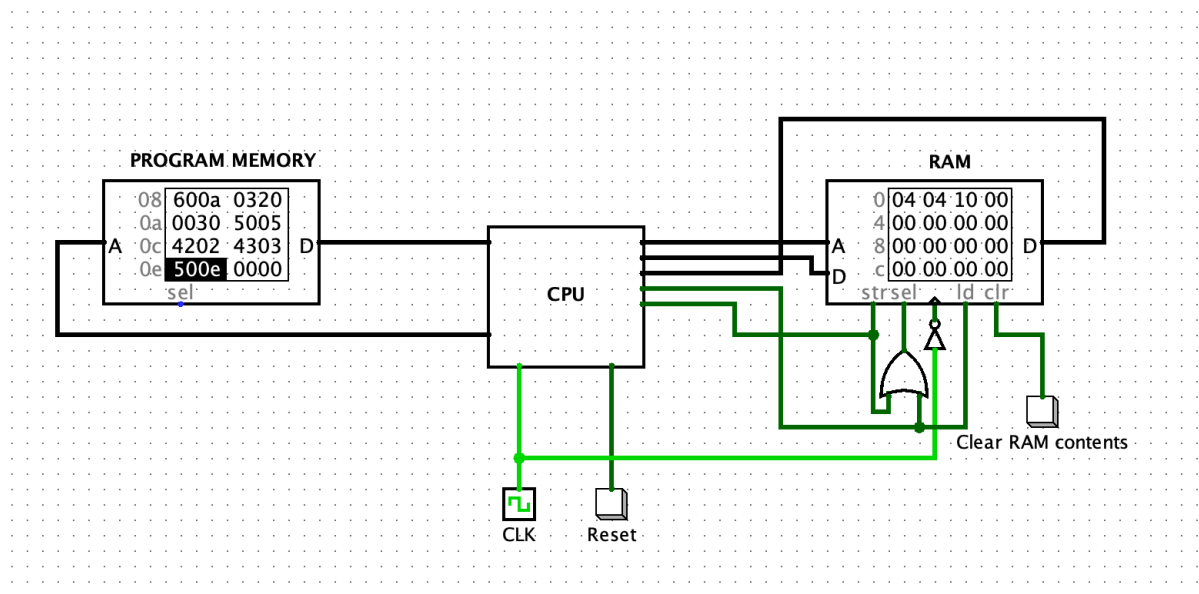
END:    JMP  END       ;0x500e stop

```

```

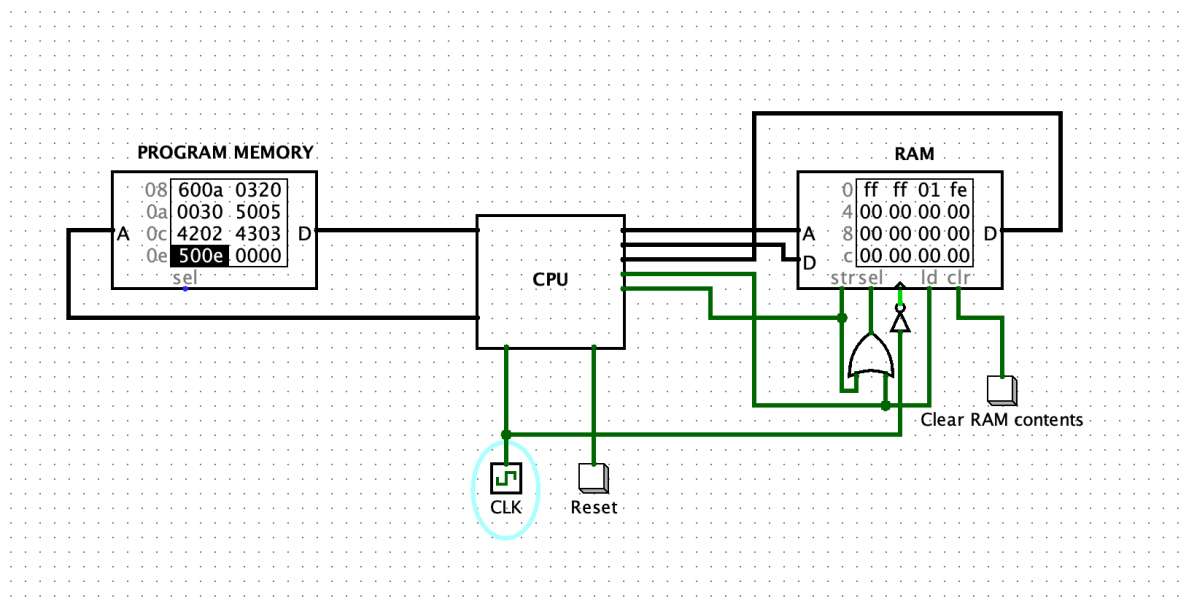
00 0090 3000 3101 2200 2300 0090 650c 0201 600a 0320 0030 5005 4202 4303 500e 0000
10 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000
20 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000
30 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000
40 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000
50 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000
60 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000
70 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000
80 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000
90 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000
a0 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000
b0 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000
c0 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000
d0 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000
e0 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000
f0 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000

```



$$4_{16} \times 4_{16}$$

$$10_{16}$$



$FF_{16} \times FF_{16}$
 $FE01_{16}$

Programma nolasa divus 8 bitu skaitļus no RAM atmiņas, veic reizināšanu ar atkārtotas saskaitīšanas metodi un saglabā rezultātu kā 16 bitu skaitli.

Rezultāts RAM atmiņā tiek saglabāts Little Endian formātā, kur zemākajā adresē atrodas zemākie 8 biti, bet augstākajā adresē augstākie 8 biti.

Darba rezultātā tika apgūta assembler programmēšana, mašīnkoda veidošana un mikroprocesora darbības pārbaude Logisim vidē.